SEPTEMBER 12, 2026

shared_ptr Is Not a Lifetime Strategy

If you add `std::shared_ptr` because you are not sure how long an object should live, you probably have not solved the lifetime problem. You have hidden it.

This happens easily in asynchronous C++.

A callback needs access to an object later. Capturing `this` looks dangerous, so somebody changes the class to inherit from `std::enable_shared_from_this` and writes:

```
scheduler.post([self = shared_from_this()] {
    self->process();
});
```

The use-after-free disappears. The code looks safer. The review passes.

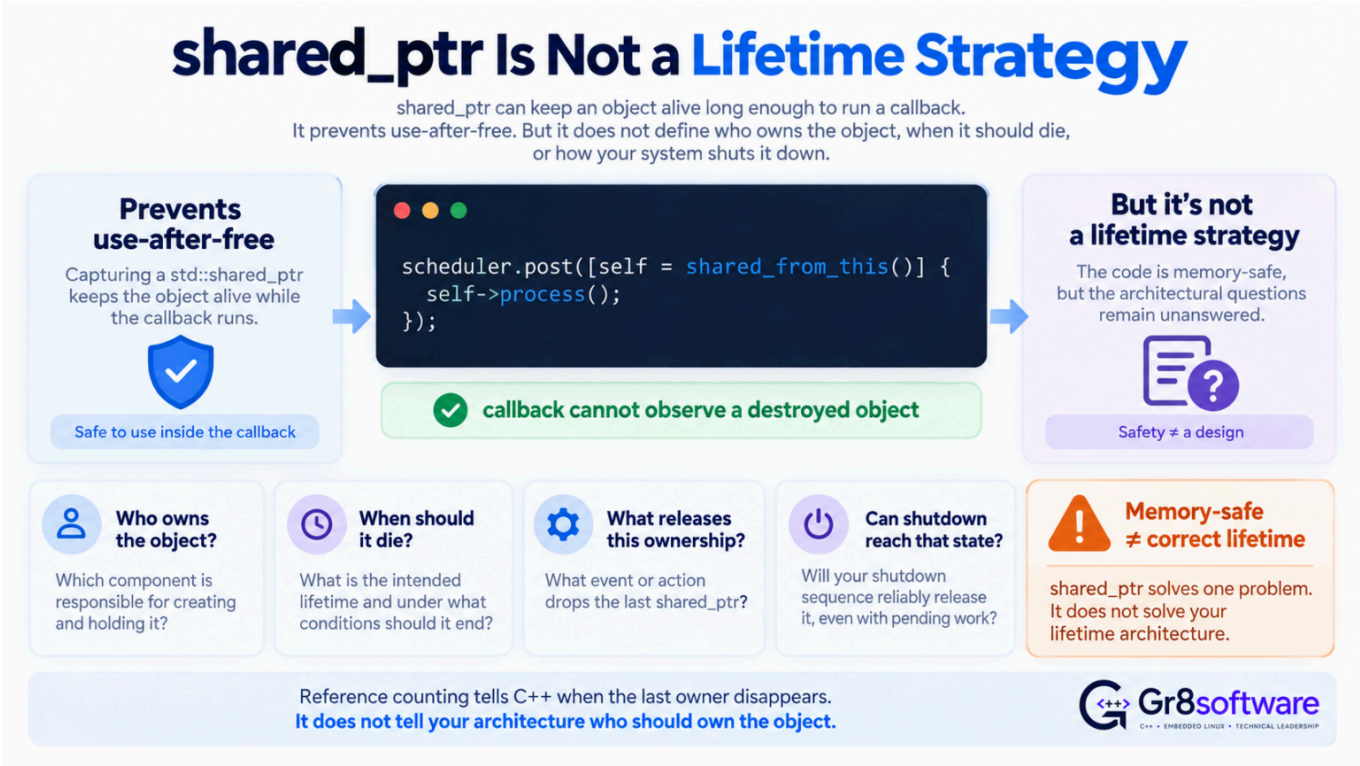
But several harder questions have just been moved somewhere else:

- **Who actually owns `self`?**
- **When is this object supposed to die?**
- **Is the callback intentionally extending its lifetime?**
- **What happens if the callback never runs?**
- **Can this reference participate in a cycle?**
- **Which thread eventually destroys the object?**
- **Can shutdown ever reach the destructor that is supposed to cancel the callback?**

Those are lifetime-design questions. Reference counting does not answer them.

> **Note**
>
> This article is not an argument against `std::shared_ptr`. Shared ownership is a legitimate design. The problem is using shared ownership as a substitute for

> deciding what the ownership model actually is.



shared_ptr solves one specific problem

`std::shared_ptr<T>` gives several owners shared responsibility for an object's lifetime.

Conceptually, every owning copy participates in the same control block:

```
auto session = std::make_shared<Session>();

auto a = session;
auto b = session;

// Session remains alive while at least one
// owning shared_ptr still exists.
```

When the final owning reference is released, the managed object is destroyed.

That mechanism is useful when multiple components genuinely need independent ownership. But it does not establish:

- which components are supposed to own the object;
- which references are merely observers;
- when the logical lifetime should end;

- how outstanding asynchronous work is cancelled;
- how shutdown is coordinated;
- where destruction is allowed to execute.

Those decisions belong to the architecture.

> **Useful mental model**
>
> `std::shared_ptr` is an ownership mechanism. A lifetime strategy explains who owns the object, which operations may keep it alive, when ownership is released, and how the system reaches that state during shutdown.

The apparently safe asynchronous callback

Consider a connection object that posts packet processing onto a worker scheduler:

```
class Connection
    : public std::enable_shared_from_this<Connection>
{
public:
    void onPacket(Packet packet)
    {
        scheduler_.post(
            [self = shared_from_this(),
             packet = std::move(packet)]() mutable {
                self->process(std::move(packet));
            });
    }

private:
    void process(Packet packet);

    Scheduler& scheduler_;
};
```

This code solves a real problem. Once the task is queued, destroying the caller's `shared_ptr<Connection>` does not leave the callback with a dangling pointer. The callback owns the connection until it finishes.

But that is not automatically the correct lifetime.

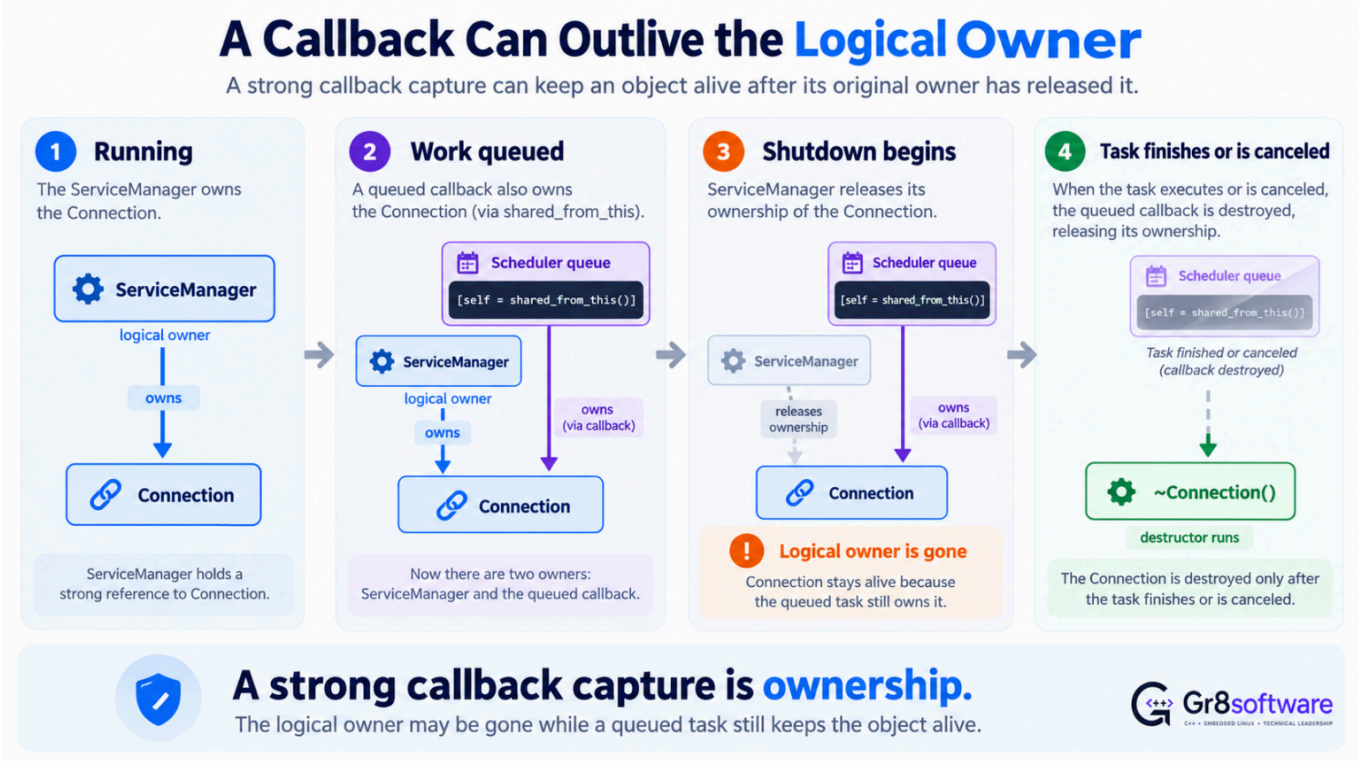
Suppose the product is shutting down. The component that created the connection logically knows there are no clients anymore. Logically, the connection should disappear.

It does not.

The scheduler queue still contains a callback. The callback contains a `shared_ptr`. The connection remains alive until the callback executes or the scheduler destroys the queued task.

If the worker is blocked, paused, stopped incorrectly or waiting behind a long queue, the object can remain alive far longer than anybody intended.

> **Important distinction**
>
> The callback is no longer merely using `Connection`. It has become one of its owners. That should be a deliberate part of the design, not an accidental side effect of avoiding a dangling pointer.



A memory-safe program can still have the wrong lifetime

This is an important production distinction.

A use-after-free is clearly wrong. Sanitizers may catch it. The application may crash. The bug has an obvious shape.

An object that lives too long is more subtle.

The process may remain completely memory-safe while still behaving incorrectly:

- a network session stays registered after its logical owner disappeared;
- a D-Bus object keeps answering calls while its subsystem is supposed to be stopping;
- a file descriptor remains open and prevents another component from acquiring a device;
- a timer continues scheduling work during shutdown;
- a middleware session keeps a large buffer graph alive;
- a destructor containing important cleanup is never reached.

None of these requires an invalid pointer.

The lifetime itself is wrong.

The destructor-driven shutdown trap

One particularly nasty design appears when the destructor is responsible for cancelling the work that is keeping the object alive.

```
class Session
    : public std::enable_shared_from_this<Session>
{
public:
    ~Session()
    {
        timer_.cancel();
    }

    void armTimeout()
    {
        timer_.asyncWait([self = shared_from_this()] {
            self->onTimeout();
        });
    }

private:
```

```
    void onTimeout();

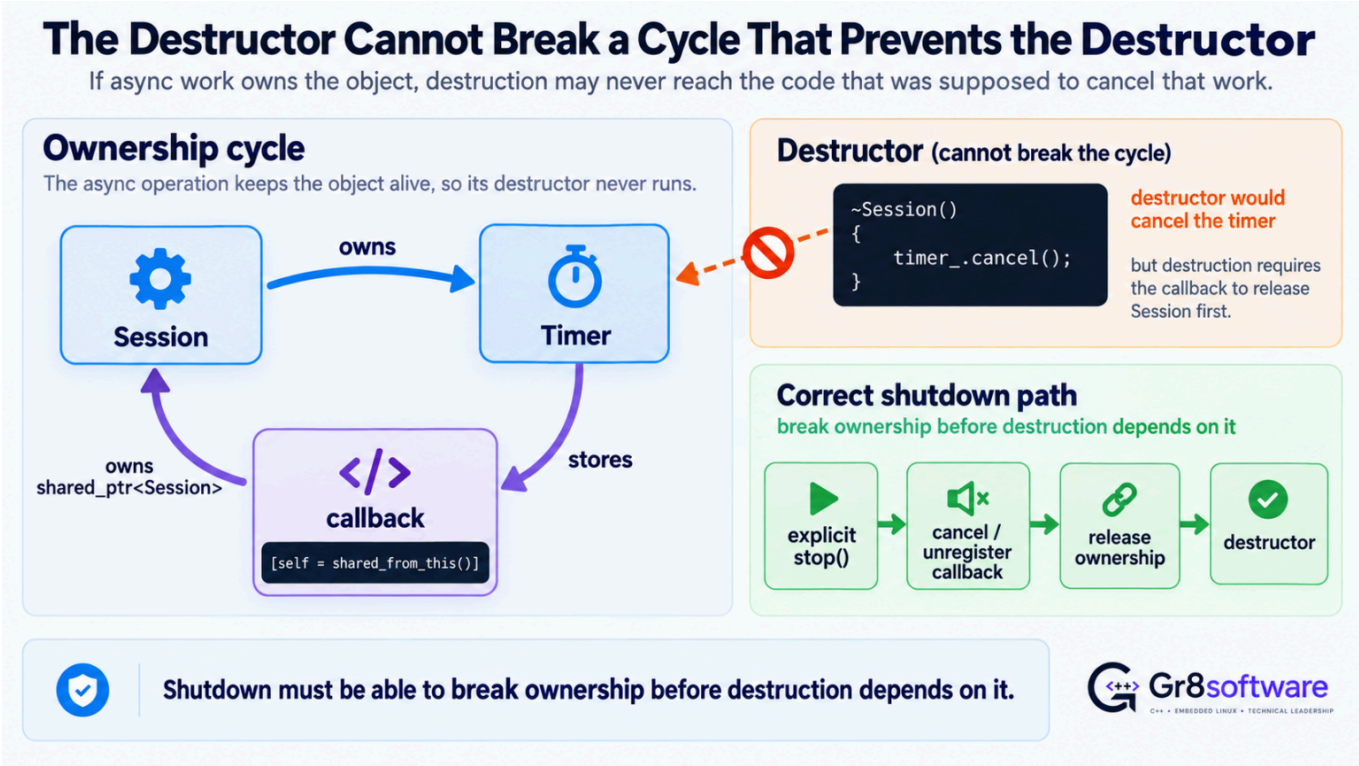
    Timer timer_;
};
```

Read the lifecycle carefully.

1. `Session` owns `timer_`.
2. The timer stores a callback.
3. The callback owns a `shared_ptr<Session>`.
4. The external owner releases its `shared_ptr<Session>`.
5. The timer callback still keeps the session alive.
6. The destructor therefore does not run.
7. But the destructor contains the call that was supposed to cancel the timer.

The cleanup cannot happen because the reference that cleanup would remove is preventing cleanup from starting.

> **Common failure mode**
>
> If asynchronous work can own the object, do not assume the object's destructor can always be the mechanism that stops that work. The destructor may be waiting indirectly for the work to release the last reference first.



Not every lifetime extension is a reference cycle

Reference cycles get most of the attention because they are easy to draw:

```
class Parent
{
    std::shared_ptr<Child> child_;
};

class Child
{
    std::shared_ptr<Parent> parent_;
};
```

Neither reference count can reach zero.

Callbacks create the same shape surprisingly easily:

```
class Controller
    : public std::enable_shared_from_this<Controller>
{
public:
    void start()
    {
        timer_.setCallback([self = shared_from_this()] {
            self->poll();
        });

        timer_.start();
    }

private:
    Timer timer_;
};
```

If `Controller` owns the timer, the timer owns the callback, and the callback owns the controller, the ownership graph closes on itself.

But an object can also live forever without a literal cycle.

Consider a global or long-lived executor:

```
void Session::scheduleRetry()
{
    executor_.post_after(
        5s,
        [self = shared_from_this()] {
            self->retry();
        });
}
```

If every retry schedules another retry holding another strong reference, the object can keep itself alive indefinitely even though the executor is external and no ownership graph points directly back through a member.

Each callback eventually releases its reference. It simply creates the next one before doing so.

Reference-count analysis alone will not reveal the design error. You have to understand the state machine.

Self-rescheduling work can become hidden ownership

This pattern appears often in production systems:

```
void TelemetryClient::scheduleReconnect()
{
    scheduler_.post_after(
        reconnectDelay_,
        [self = shared_from_this()] {
            if (self->connect()) {
                self->startMonitoring();
                return;
            }
```

```
            self->scheduleReconnect();
        });
}
```

The intention is reasonable. The client should keep retrying while the remote service is unavailable.

But what ends that policy?

- Does releasing the original `shared_ptr` stop retries? No.
- Does entering system shutdown stop retries? Not unless the callback checks shutdown state.
- Does the destructor cancel them? The destructor may not run.
- Can another component know whether retries still exist? Perhaps not.

The callback has silently become the lifetime controller for the entire client.

> **Design question**
>
> Whenever asynchronous work captures a strong pointer to its parent object, ask what event eventually removes that ownership. “When the callback executes” is not sufficient if executing the callback creates another callback.

weak_ptr changes the contract

A common alternative is to capture a `std::weak_ptr`:

```
scheduler_.post(
    [weak = weak_from_this()] {
        if (auto self = weak.lock()) {
            self->process();
        }
    });
```

This does something very different from capturing `shared_from_this()`.

The callback no longer says:

“This work requires the object to remain alive until I run.”

It says:

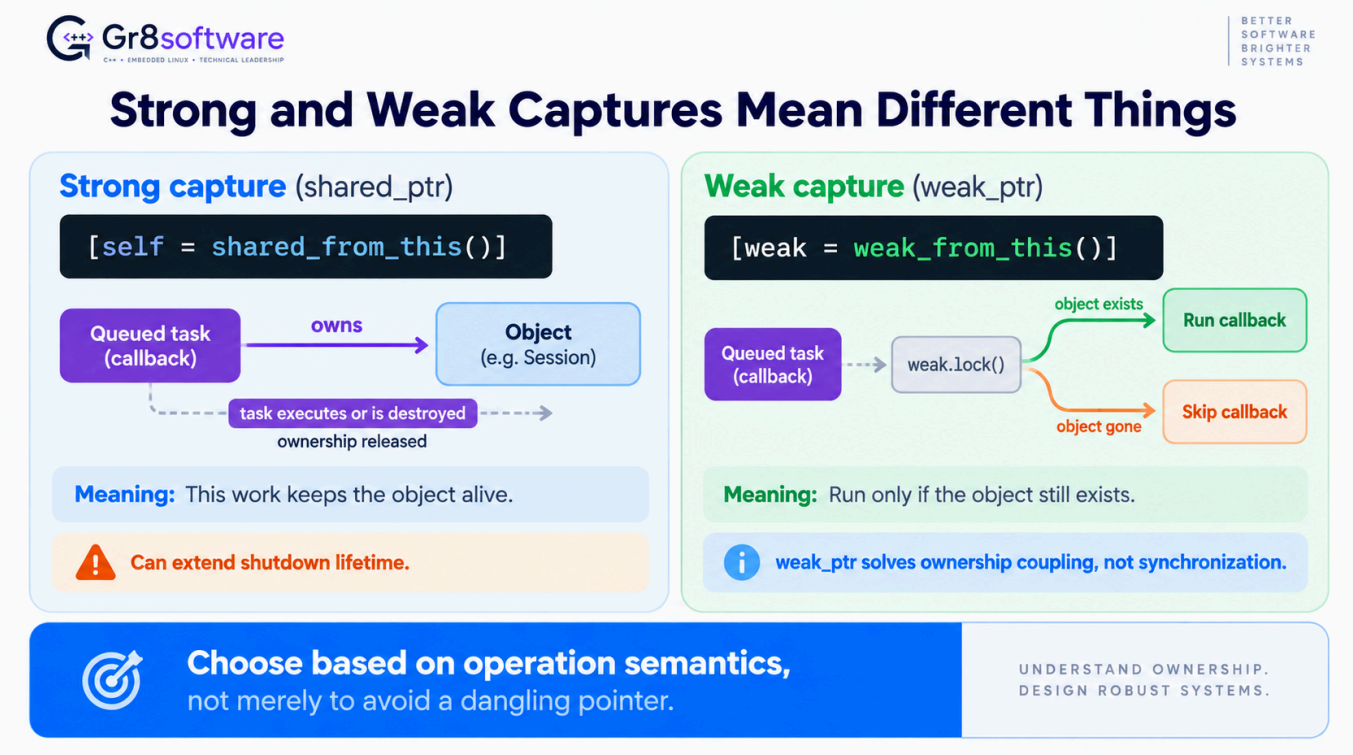
“When I eventually run, perform the work only if the object still exists.”

That may be exactly the semantics you need for a notification, UI update, optional timeout or stale event.

It may also be completely wrong.

If the queued operation represents work that must finish — committing persistent state, completing a protocol transaction or replying to a request — silently dropping it because the parent disappeared may violate a different contract.

> **Do not mechanically replace shared_ptr with weak_ptr**
>
> `weak_ptr` is not the “safe async pointer”. It expresses non-ownership. Use it when skipping the callback after the object’s lifetime ends is valid behaviour.



weak_ptr solves ownership, not synchronization

Another mistake is assuming that successfully locking a `weak_ptr` makes the object safe to use concurrently.

```
if (auto self = weak.lock()) {
    self->state_ = State::Running;
}
```

The strong pointer guarantees that the object will not be destroyed while `self` exists.

It says nothing about another thread simultaneously reading or modifying `state_`.

Object lifetime and thread safety are separate properties. You may still need:

- a mutex;
- an executor or strand that serializes access;
- atomics;
- message passing;
- or an explicit state-machine rule that all mutations happen on one thread.

Reference-count operations being safe across threads does not make the managed object internally thread-safe.

The last shared_ptr decides where destruction runs

This consequence deserves more attention in asynchronous software.

The object is normally destroyed synchronously by whichever thread releases the final owning `shared_ptr`.

Consider:

```
scheduler_.post(
    [self = shared_from_this()] {
        self->process();
    });
```

Suppose the main thread releases the original owner while this callback is queued. The callback now owns the final strong reference.

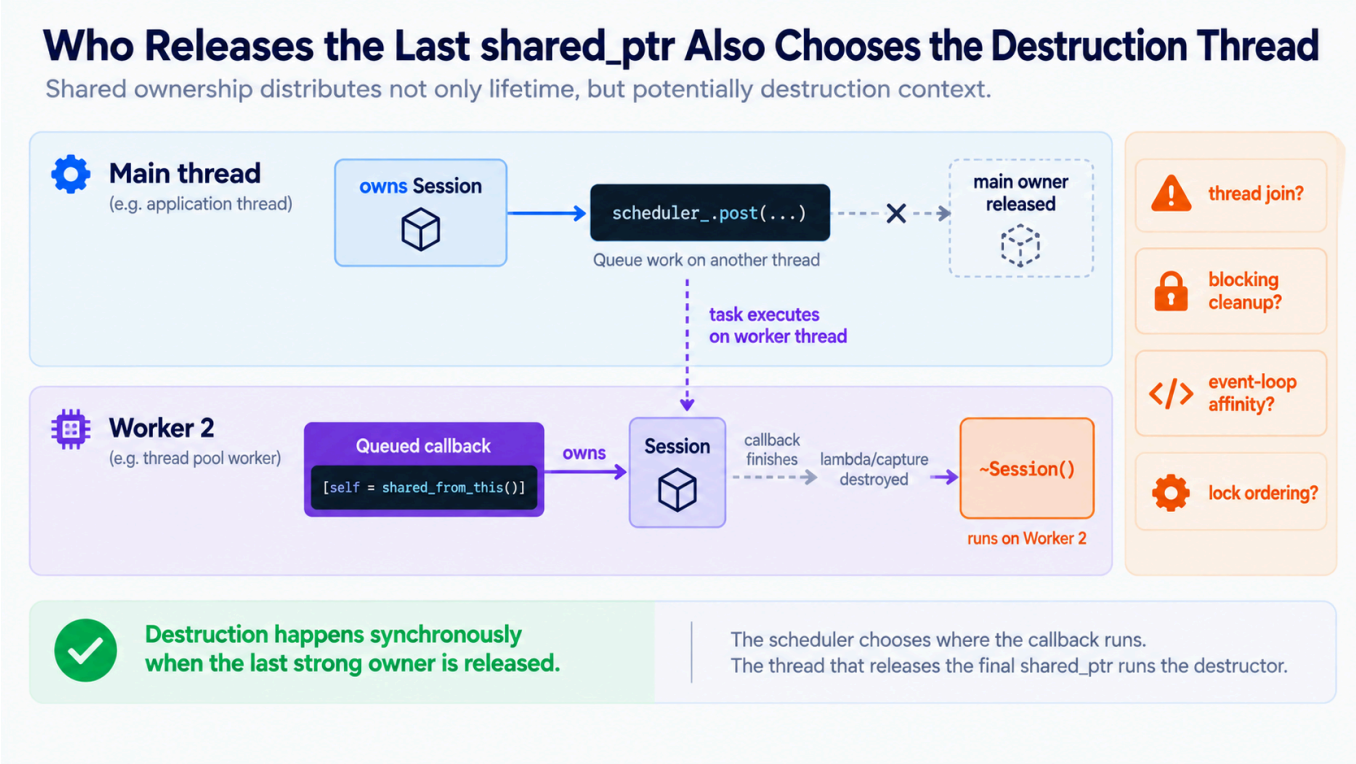
When the worker finishes and its lambda is destroyed, the reference count reaches zero. The `Session` destructor can therefore execute on the worker thread.

That can be a serious surprise if destruction:

- blocks waiting for other workers;
- joins a thread;
- deregisters objects from a thread-affine event loop;
- closes hardware resources expected to be touched only from one execution context;
- flushes a large amount of state;
- takes locks in an order that the worker path did not expect.

A destructor that joins the current worker thread is an extreme example. Less dramatic cases show up as shutdown latency spikes or rare deadlocks.

> **Production consequence**
>
> With distributed shared ownership, you do not only lose clarity about *when* destruction happens. You can also lose control over *where* it happens.



shared_from_this has its own failure modes

`std::enable_shared_from_this` is useful, but it requires the object to participate correctly in a `shared_ptr` control block.

This is valid:

```
auto session = std::make_shared<Session>();
session->start();
```

This is not the same ownership model:

```
Session session;
session.start();
```

If `start()` calls `shared_from_this()`, the object is not owned by the required `shared_ptr` control block. A call to `shared_from_this()` in that state fails with `std::bad_weak_ptr`.

Calling it from the constructor is another classic trap:

```
class Session
    : public std::enable_shared_from_this<Session>
{
public:
    Session()
    {
        scheduler_.post(
            [self = shared_from_this()] {
                self->initializeAsync();
            });
    }
};
```

The enclosing `shared_ptr` has not finished establishing the relationship needed by `enable_shared_from_this` while the constructor is executing.

A factory plus an explicit second initialization phase is usually clearer:

```
class Session
    : public std::enable_shared_from_this<Session>
{
public:
```

```
    static std::shared_ptr<Session> create(Scheduler& scheduler)
    {
        auto session =
            std::shared_ptr<Session>(
                new Session(scheduler));

        session->start();
        return session;
    }

private:
    explicit Session(Scheduler& scheduler)
        : scheduler_(scheduler)
    {
    }

    void start()
    {
        scheduler_.post(
            [self = shared_from_this()] {
                self->initializeAsync();
            });
    }

    Scheduler& scheduler_;
};
```

The exact factory shape is project-specific. The important part is separating construction from operations that require shared ownership.

Never create another shared_ptr from this

A particularly dangerous attempt to solve callback lifetime looks like this:

```
scheduler_.post(
    [self = std::shared_ptr<Session>(this)] {
        self->process();
    });
```

Do not do this.

If the object is already managed by another `shared_ptr`, this normally creates a second independent control block for the same raw pointer.

Now two reference counts believe that they independently own the same object. Eventually both can attempt to delete it.

> **Rule**
>
> If a type intentionally supports obtaining shared ownership of itself, use `std::enable_shared_from_this` and construct it through the ownership model designed for that type. Do not manufacture a new owning `shared_ptr` from `this`.

An explicit owner is often easier to reason about

Many components do not need distributed ownership at all.

Imagine a product service manager that creates a telemetry component, starts it, later stops it, waits for outstanding work and finally destroys it.

The ownership can be simple:

```
class ServiceManager
{
public:
    void start()
    {
        telemetry_ =
            std::make_unique<TelemetryService>(scheduler_);

        telemetry_->start();
    }

    void stop()
    {
        telemetry_->stop();
        scheduler_.drain();
```

```
        telemetry_.reset();
    }

private:
    Scheduler scheduler_;
    std::unique_ptr<TelemetryService> telemetry_;
};
```

This example assumes that the scheduler's drain contract really guarantees that callbacks using the service have completed or have been removed before `telemetry_.reset()`. Real executors differ, so the exact API will vary.

The important property is architectural:

- there is one clear owner;
- shutdown starts explicitly rather than accidentally through destruction;
- new asynchronous work is stopped;
- outstanding callbacks are cancelled or drained;
- only then is the object destroyed.

With that contract in place, an asynchronous callback capturing raw `this` can even be correct:

```
void TelemetryService::scheduleWork()
{
    scheduler_.post([this] {
        processNextBatch();
    });
}
```

The safety does not come from the pointer type. It comes from the lifecycle guarantee that the owner will not destroy `TelemetryService` until callbacks have been drained or cancelled.

> **Do not copy this pattern without the guarantee**

> A raw `this` in delayed work is safe only when the surrounding execution model provides a strong and reviewed lifetime rule. Without that rule, it is still a use-after-free waiting to happen.

Sometimes the operation should own state, not the whole service

Another useful design is to separate the lifetime of an asynchronous operation from the lifetime of the large object that started it.

Suppose a diagnostic service launches a request that needs a payload, transport handle and completion state. Keeping the entire service alive may be unnecessary.

```
struct RequestState
{
    RequestId id;
    Packet request;
    std::shared_ptr<Transport> transport;

    void execute()
    {
        transport->send(request);
    }
};

void DiagnosticService::send(Packet request)
{
    auto state = std::make_shared<RequestState>(
        nextRequestId(),
        std::move(request),
        transport_);

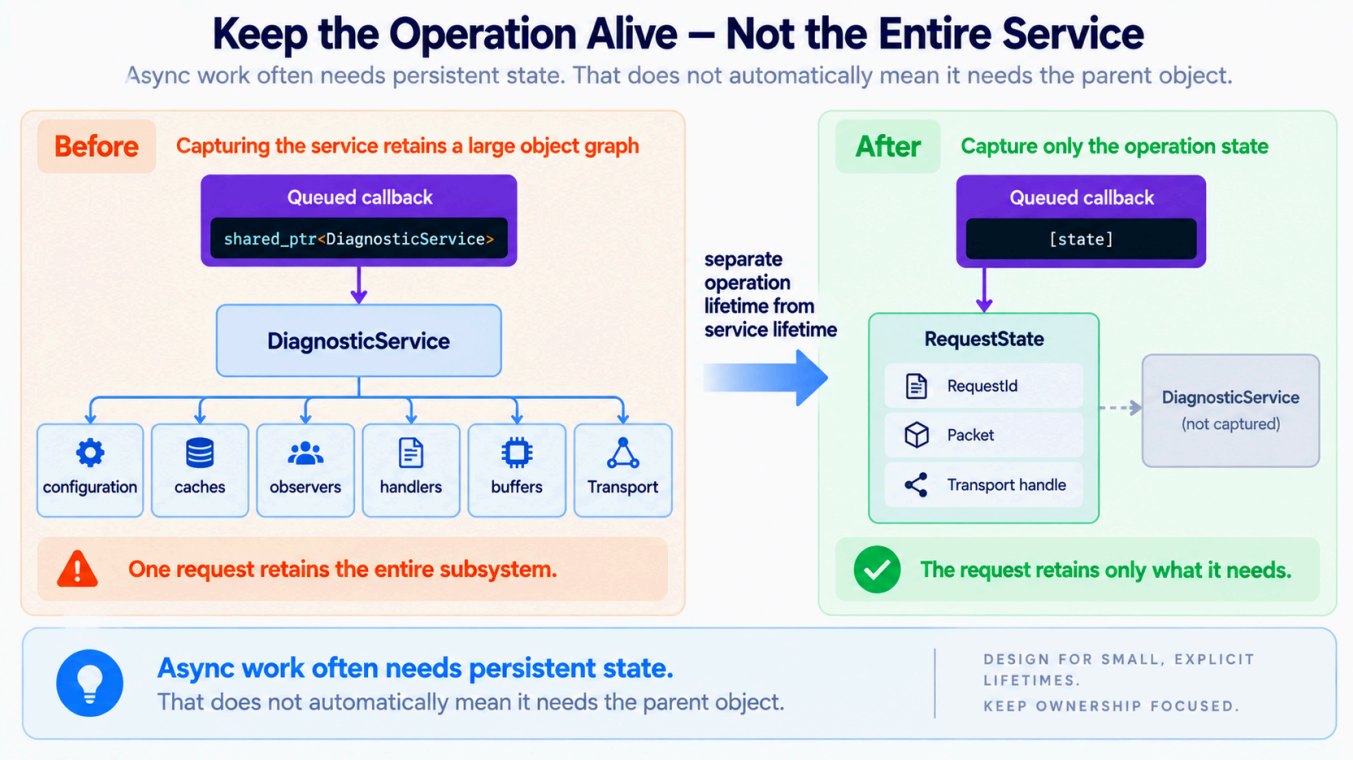
    scheduler_.post([state] {
        state->execute();
    });
}
```

Now the callback keeps the operation state alive, not the entire `DiagnosticService`.

That can make the ownership graph much smaller and easier to reason about.

If the scheduler supports move-only callables and only one task owns the state, even shared ownership may be unnecessary. The operation can be moved directly into the callback.

> **Production recommendation**
>
> When one asynchronous operation needs to outlive its originating method, ask whether the operation itself can own the required state instead of extending the lifetime of an entire subsystem object.



Explicit stop is often better than hoping destruction means stop

For long-running asynchronous objects, I prefer shutdown to be visible in the API.

```
class Client
{
public:
    void start();
    void stop();
```

```
private:
    bool stopping_{false};
};
```

The exact implementation can use a state machine, cancellation source, executor synchronization, event-loop posting or another project-specific mechanism.

A typical shutdown sequence is conceptually:

1. Reject or stop creating new work.
2. Signal cancellation to outstanding work.
3. Unregister callbacks, timers and observers.
4. Wait for work that must complete, or explicitly abandon work that may be dropped.
5. Release resources that depend on the execution context.
6. Destroy the object.

The destructor can still contain defensive cleanup. It should not necessarily be the first time the system tries to coordinate shutdown.

shared_ptr is correct when ownership really is shared

There are systems where several independent actors genuinely need to keep one object alive.

For example:

- multiple asynchronous operations use the same immutable resource;
- a cache and active clients independently own the same object;
- an object represents shared immutable configuration snapshots;
- a runtime intentionally distributes ownership between subsystems with no single natural parent.

In those cases, `std::shared_ptr` can express the architecture honestly.

The difference is that you should be able to describe the ownership policy without mentioning reference counts:

“The session stays alive while it is registered in the connection table or while an active request is using it. Shutdown first removes it from the table, prevents new requests, cancels pending operations and waits until active requests release their ownership.”

That is a lifetime strategy.

"It uses `shared_ptr` so it should stay alive somehow" is not.

Make ownership visible in APIs

The pointer type passed through an API should match the relationship that API needs.

Relationship	Typical representation	Meaning
Sole ownership	`std::unique_ptr<T>`	One owner controls lifetime
Use existing object	`T&` / `const T&`	No ownership transfer
Optional observation	`T*`	Non-owning, may be absent
Participate in shared ownership	`std::shared_ptr<T>`	This code may extend lifetim
Observe shared object if still alive	`std::weak_ptr<T>`	Does not extend lifetime
Async operation owns only required state	Dedicated operation state	Operation lifetime is separate subsystem lifetime

A function that merely uses a transport does not need this:

```
void encode(
    const std::shared_ptr<Transport>& transport);
```

If ownership is irrelevant, take the object:

```
void encode(const Transport& transport);
```

If the callee intentionally stores shared ownership, make that visible:

```
void setTransport(std::shared_ptr<Transport> transport)
{
```

```
    transport_ = std::move(transport);
}
```

Passing the smart pointer should mean something.

Reference counting also has a runtime cost

Lifetime clarity is the main issue here, but there is also a performance consequence.

Copies of a `shared_ptr` manipulate the ownership count in a control block designed to support ownership activity from multiple threads. In heavily contended paths, repeatedly copying shared pointers can create synchronization and cache-coherency traffic.

Code such as this can therefore have more cost than the syntax suggests:

```
for (const auto& packet : packets) {
    queue.post([session] {
        session->process();
    });
}
```

Every callback obtains another owning reference.

In ordinary control-plane code the cost may be irrelevant. In high-rate messaging, media processing, networking or constrained embedded software, it is worth measuring.

> **Do not optimize the wrong problem**
>
> Replacing `shared_ptr` merely because reference counting has overhead is usually the wrong starting point. First establish a correct ownership model. Then measure whether ownership traffic is actually important in your workload.

Do not use use_count() as a lifetime protocol

When debugging shared ownership, it is tempting to start making decisions from `use_count()`:

```
if (session.use_count() == 1) {
    // I must be the final owner...
}
```

That is not a useful synchronization protocol.

The count is only a snapshot. Other owners may appear or disappear immediately around that observation. Even in single-threaded code, basing architecture on a magic reference count usually means the ownership relationships are not explicit enough.

`use_count()` can occasionally help while instrumenting a problem. It should not decide whether an operation is safe, whether shutdown may proceed or who owns the object conceptually.

Debugging objects that refuse to die

A shared-ownership lifetime bug often looks different from a traditional memory leak.

Start with simple instrumentation. Give important long-lived objects an ID and log both creation and destruction:

```
Session::Session(SessionId id)
    : id_(id)
{
    logDebug("Session {} constructed", id_);
}

Session::~Session()
{
    logDebug(
        "Session {} destroyed on thread {}",
        id_,
        std::this_thread::get_id());
}
```

Then instrument lifecycle boundaries rather than every pointer copy:

- registration and unregistration from observers;
- timer creation and cancellation;

- queued callbacks;
- start and stop transitions;
- creation and completion of asynchronous operations.

If shutdown says “session removed” but the destructor never appears, ask what still owns it.

Sanitizers help, but semantic retention can remain invisible

AddressSanitizer is excellent for the opposite lifetime bug: using an object after destruction.

```
-fsanitize=address,undefined -fno-omit-frame-pointer
```
terminal

On suitable Linux builds, leak detection can also reveal unreachable allocations at process exit:

```
ASAN_OPTIONS=detect_leaks=1 ./device-daemon
```
terminal

A pure reference cycle may show up as leaked memory.

But consider a global event dispatcher that legitimately remains reachable until process exit and stores a callback containing your session’s `shared_ptr`. The allocator may see perfectly reachable memory. From its perspective there is no leak.

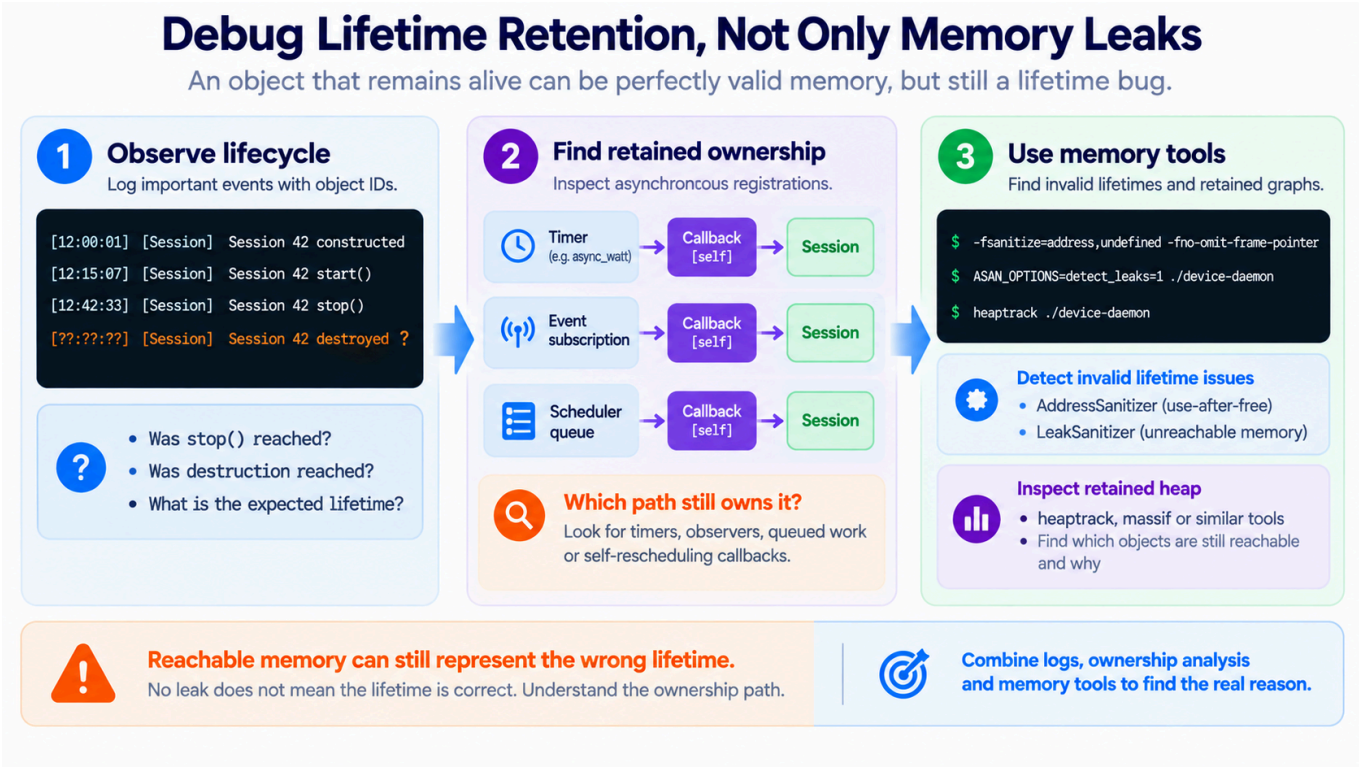
From your product’s perspective, a session that should have disappeared thirty minutes ago is still alive.

That is why lifetime debugging needs architectural observability in addition to memory tools.

Heap profiling can still help when retained object graphs are large. On Linux, for example:

```
heaptrack ./device-daemon
```
terminal

The useful question is not only “what allocation was never freed?” but also “why does this subsystem still have a reachable ownership path?”



A production example: event subscription

Event buses and observer registries are another common source of hidden ownership.

```
void DeviceController::start()
{
    events_.subscribe(
        DeviceEvent::Changed,
        [self = shared_from_this()](const Event& event) {
            self->onDeviceChanged(event);
        });
}
```

If `events_` is a process-wide dispatcher, this subscription can keep `DeviceController` alive until the dispatcher removes the callback.

Releasing the controller everywhere else does nothing.

A better API often returns an explicit subscription handle:

```
subscription_ = events_.subscribe(
    DeviceEvent::Changed,
    [weak = weak_from_this()](const Event& event) {
```

```
        if (auto self = weak.lock()) {
            self->onDeviceChanged(event);
        }
    });
```

Then shutdown has something concrete to do:

```
void DeviceController::stop()
{
    subscription_.reset();
}
```

Here two independent decisions are visible:

- the subscription handle controls whether events remain registered;
- the callback itself does not own the controller.

That is much easier to reason about than hoping reference counts eventually fall to zero.

The questions I would ask in code review

Whenever I see a new `shared_ptr` capture or member in asynchronous code, these are the questions worth answering:

- **Who is the conceptual owner?** Can we name the component responsible for ending this object's lifetime?
- **Why does this code need ownership?** Would a reference, pointer, weak reference or dedicated operation state express the relationship better?
- **What event releases this ownership?** Callback completion, cancellation, unsubscription, connection close, explicit stop?
- **Can that event fail to happen?** What if the event loop stops, a timer remains pending or the remote side never replies?
- **Can the reference close a cycle?** Include callbacks stored inside owned children, not only direct pointer members.
- **What does shutdown do first?** Stop new work, cancel, drain, unregister or just release a pointer?
- **Where can the last owner disappear?** Main thread, worker, I/O thread, callback destructor?
- **Is destruction safe on that thread?**
- **If we use weak_ptr, is dropping the operation actually allowed?**

- **Does the object need shared ownership at all?**

A practical ownership decision table

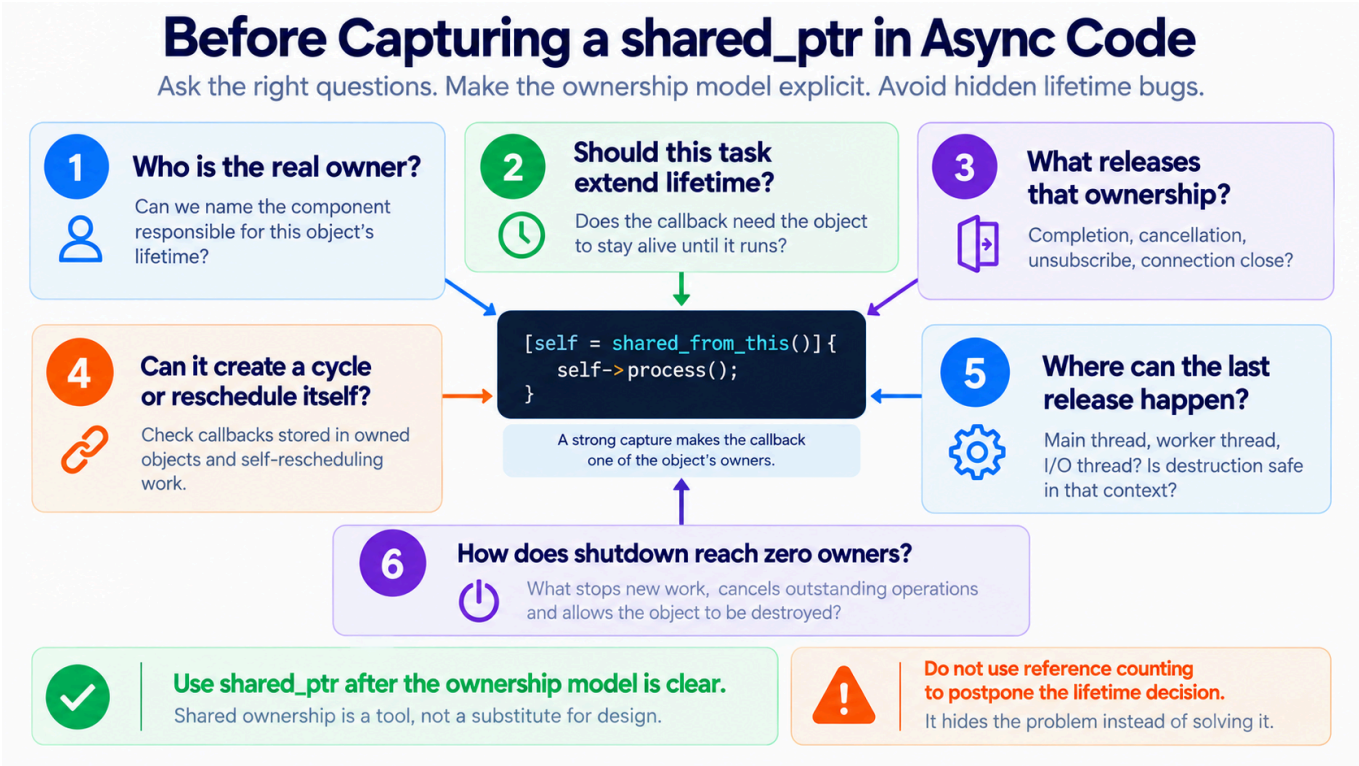
Situation	Good starting point
One subsystem clearly owns the object	`std::unique_ptr` plus explicit shutdo
Callback is guaranteed not to outlive owner because executor is drained before destruction	Non-owning capture may be sufficient
Callback may run only while object still exists	`std::weak_ptr` and `lock()`
Callback must keep object alive until operation finishes	Strong ownership may be correct, but c cancellation and shutdown
Only operation data must survive	Dedicated operation-state object
Several independent components genuinely own one object	`std::shared_ptr` with a documented ownership policy
Parent and child point at each other	Choose one ownership direction; make back-reference non-owning
Self-rescheduling callback retains the object indefinitely	Explicit stop state and cancellation; do on destructor-triggered shutdown

Production recommendations

A few rules make asynchronous ownership much easier to reason about.

- **Start with the owner, not the smart pointer.** Decide which component controls the logical lifetime before choosing `unique_ptr`, `shared_ptr` or `weak_ptr`.
- **Treat every strong callback capture as ownership.** A lambda containing `shared_ptr` is another owner even if nobody describes it that way in the class diagram.
- **Make shutdown explicit for long-running async objects.** Stop producing work, cancel or drain existing operations, then destroy the object.

- **Use weak_ptr only when disappearing work is acceptable.** It expresses conditional access, not guaranteed completion.
- **Keep operation lifetime separate where possible.** Do not retain an entire service object when a small request-state object contains everything the asynchronous operation needs.
- **Review destruction context.** With shared ownership, the last release can happen in a worker callback. Destructors should not contain surprising thread-affine or blocking behaviour unless the architecture guarantees the execution context.
- **Do not put critical cancellation exclusively in the destructor when callbacks own self.** The destructor may not be reachable until those callbacks are cancelled.
- **Instrument lifecycle boundaries.** Construction, start, stop, registration, cancellation and destruction logs are often more useful than logging raw reference counts.
- **Measure shared ownership in hot paths.** Correctness comes first, but repeated reference-count traffic can matter in high-rate systems.



Conclusion

`std::shared_ptr` is good at keeping an object alive while somebody still owns it.

That sounds obvious, but asynchronous code makes the word *somebody* surprisingly complicated. It may be a scheduler queue, timer, event subscription, retry callback, request object or worker thread that nobody thinks of as an owner.

Once that happens, fixing a dangling pointer can easily create a different class of bug: objects that outlive their logical role, shutdown sequences that never finish, cycles that never reach zero and destructors that execute on unexpected threads.

The useful question is therefore not:

“How do I make sure this callback still has an object?”

Ask instead:

“Should this callback own the object at all, and what exact event is supposed to end that ownership?”

If that answer is clear, `shared_ptr` can be a very useful implementation tool.

If it is not clear, another `shared_ptr` usually makes the uncertainty harder to see.

Adam Bieńkuński

Lead C++ & Embedded Linux Engineer

Email Privacy Policy